

Lab 4 and Testing

Overview

Lab 4 implements security across the application, building on the foundations established in Labs 1-3. It introduces field-level encryption, database encryption, agent threat detection, code hardening, and unit tests validating security functionality.

Security & Encryption

- PHI encrypted with AES-256/GCM before every Room write - ciphertext visible on patient record screen confirms this is working. I have had issues getting this information decrypted.
- Account lockout after 3 failed logins reduces brute-force attack surface on admin credentials
- R8/ProGuard obfuscation renames classes and methods in release builds

Compliance

- GDPR Article 32 - encryption at rest (SQLCipher, AES-256/GCM) as an appropriate technical measure for PHI protection
- NHS Data Security and Protection Toolkit (DSPT) - recordedBy and updatedAt fields on EHR records, RBAC enforced at Activity and Fragment level, PHI encrypted at rest

Unit Tests

Tests are located in

test/java/com/example/hospimanagmenetapp/SecurityUnitTest.java (Figure 1)

- NHS Mod 11 validation - valid number, invalid check digit, null input, spaces in input, too short (Figure 2)
- SHA-256 hashing- same input produces same hash, different inputs produce different hashes, output does not equal input (Figure 3)
- Input sanitisation - SQL keywords removed, HTML characters removed, semicolons removed, null input returns empty string, clean input is unchanged (Figure 4)

Dynamic Testing

Lab 1:

Admin registration and login (Figure 5 - 10)

Invalid login credentials (Figure 11)

Register a patient (Figures 12 - 14)

Try invalid NHS number (Figures 14 -16)

Lab2:

Biometric scan to open appointments (Figures 17 - 18)

Filtering appointments (Figures 19 -21)

Adding valid and invalid appointments (Figures 22 - 23)

Lab 3:

Barcode scanner (Figures 24 - 28)

View patient summary and edit clinical records (Figures 28 - 31)

Record vitals and check DB via App Inspection (Figures 32 - 36)

Lab 4:

Clinical records showing encrypted data (Figures 38 - 41)

Account lockout for failed login attempts (Figures 43 - 44)

Known Limitations

- PIN hashing has no salt - two admins with the same PIN will have the same hash. Production systems should use BCrypt or PBKDF2 with a unique per-user salt
- ThreatMonitor lockout is reset by restarting the app as this resets the failure counter. A persistent lockout mechanism would need to be implemented.
- SQL encryption currently is not being decrypted properly, so when it is activated patient records show the hashed values.

Appendix – Testing images

✓ 13 tests passed 13 tests total, 10ms
Executing tasks: [:app:testDebugUnitTest, --tests, com.example.hospinaganmenetapp.SecurityUnitTest] in project C:\Users\tiann\StudioProjects\HospiManagmenetApp

Figure 1 Unit tests

```
public class SecurityUnitTest {  
  
    // NHS Number Validation  
  
    @Test  
    public void validNhsNumber_returnsTrue() {  
        assertTrue(ValidationUtils.validateNhsNumber("0123456789"));  
    }  
  
    @Test  
    public void invalidNhsNumber_wrongCheckDigit_returnsFalse() {  
        assertFalse(ValidationUtils.validateNhsNumber("0101010101"));  
    }  
  
    @Test  
    public void nhsNumber_withSpaces_isStrippedAndValidated() {  
        assertTrue(ValidationUtils.validateNhsNumber("012 345 6789"));  
    }  
  
    @Test  
    public void nullNhsNumber_returnsFalse() {  
        assertFalse(ValidationUtils.validateNhsNumber(null));  
    }  
  
    @Test  
    public void nhsNumber_tooShort_returnsFalse() {  
        assertFalse(ValidationUtils.validateNhsNumber("123456789"));  
    }  
}
```

Figure 2 NHS Number Validation

```
// PIN Hashing

@Test
public void sha256_sameInput_producesSameHash() {
    String h1 = ValidationUtils.sha256(plainPin: "1234");
    String h2 = ValidationUtils.sha256(plainPin: "1234");
    assertEquals(h1, h2);
}

@Test
public void sha256_differentInputs_produceDifferentHashes() {
    assertNotEquals(
        ValidationUtils.sha256(plainPin: "1234"),
        ValidationUtils.sha256(plainPin: "1235")
    );
}

@Test
public void sha256_isNotReversible() {
    // Hash does not equal input
    String pin = "9999";
    assertNotEquals(pin, ValidationUtils.sha256(pin));
}
```

Figure 3 Hashing Validation

```
// Input Sanitisation

@Test
public void sanitise_removesHtmlTags() {
    assertEquals( expected: "script", ValidationUtils.sanitiseInput("<script>"));
}

@Test
public void sanitise_removesSqlKeywords() {
    assertEquals( expected: "patients", ValidationUtils.sanitiseInput("DROP patients"));
}

@Test
public void sanitise_removesSemicolons() {
    assertFalse(ValidationUtils.sanitiseInput("value; DROP TABLE").contains(";"));
}

@Test
public void sanitise_nullInput_returnsEmptyString() {
    assertEquals( expected: "", ValidationUtils.sanitiseInput(null));
}

@Test
public void sanitise_cleanInput_isUnchanged() {
    assertEquals( expected: "Clean input", ValidationUtils.sanitiseInput("Clean input"));
}
}
```

Figure 4 Input Sanitisation Validation

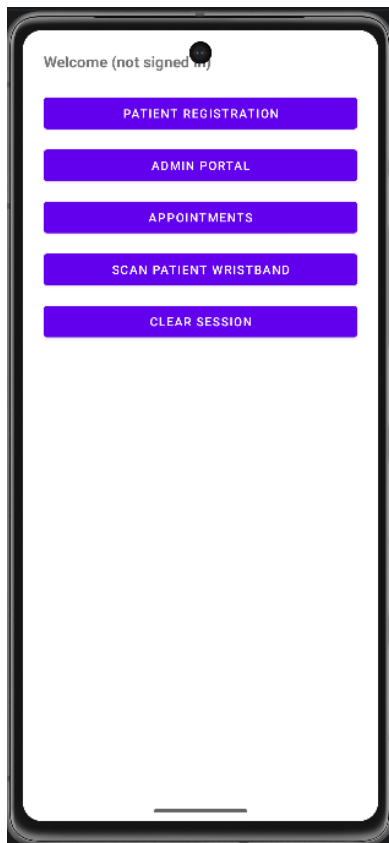


Figure 5 Admin registration

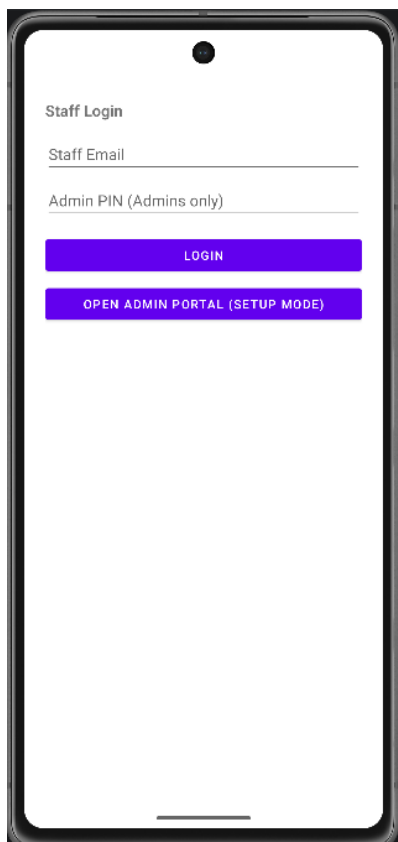


Figure 6 Admin registration

Admin Portal – Staff Registration

Full Name

Email

ADMIN

Admin PIN (required if role = ADMIN)

REGISTER STAFF

REFRESH LIST

Registered Staff

Figure 7 Admin registration

Admin Portal – Staff Registration

Full Name

Email

ADMIN

Admin PIN (required if role = ADMIN)

REGISTER STAFF

REFRESH LIST

Registered Staff

admin
admin@admin.com
ADMIN

Figure 8 Admin registration

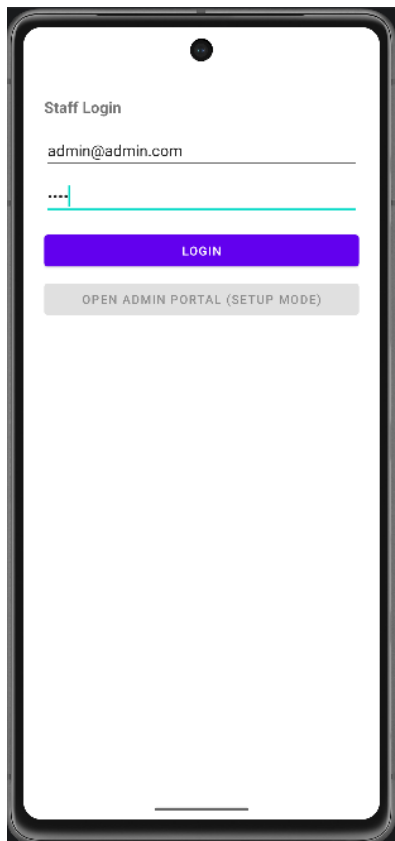


Figure 9 Login

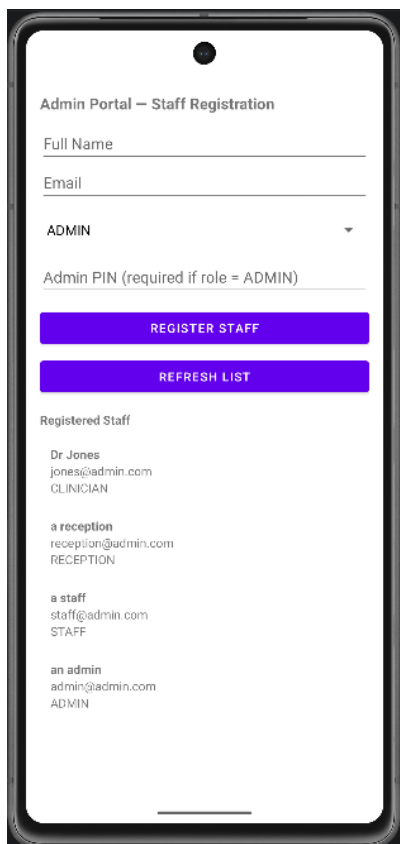


Figure 10 Login Success

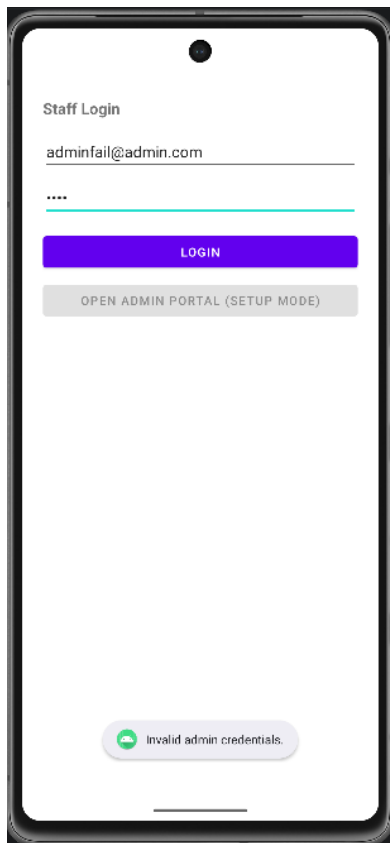


Figure 11 Invalid login

9434765919

Full Name

01-01-2001

07770124455

full.name@admin.com

SAVE PATIENT

VIEW PATIENT RECORD

Figure 12 Patient registration

Signed in: admin@adminh.com (ADMIN)

PATIENT REGISTRATION

ADMIN PORTAL

APPOINTMENTS

SCAN PATIENT WRISTBAND

CLEAR SESSION



Patient saved.

Figure 13 patient registration success

9434765919

Full Name

01-01-2001

07770124455

full.name@admin.com

SAVE PATIENT

VIEW PATIENT RECORD

Patient with this NHS number already exists.

Figure 14 Invalid login attempt

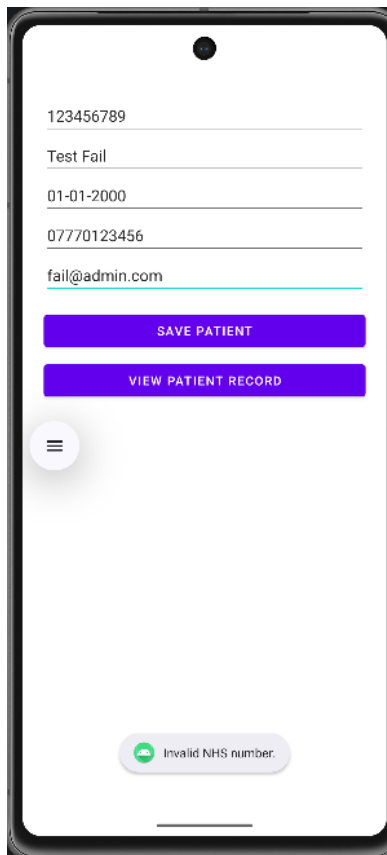


Figure 15 Invalid login attempt

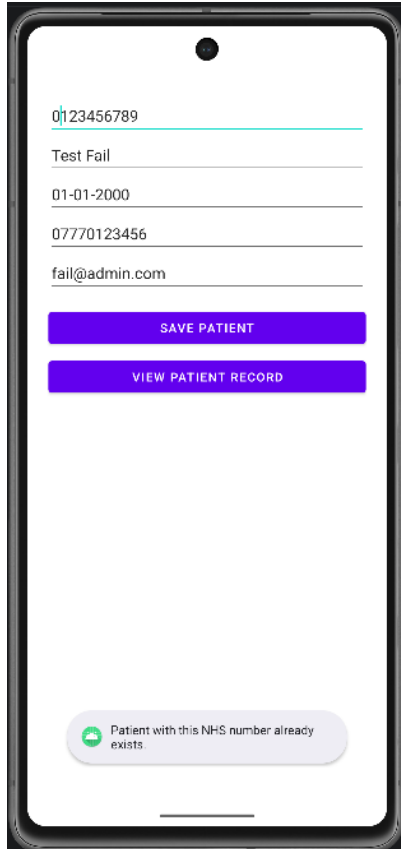


Figure 16 Invalid login attempt

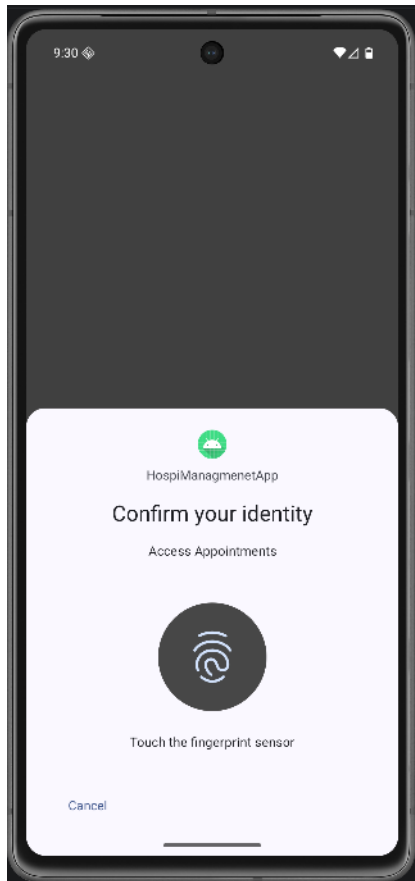


Figure 17 Biometric scan

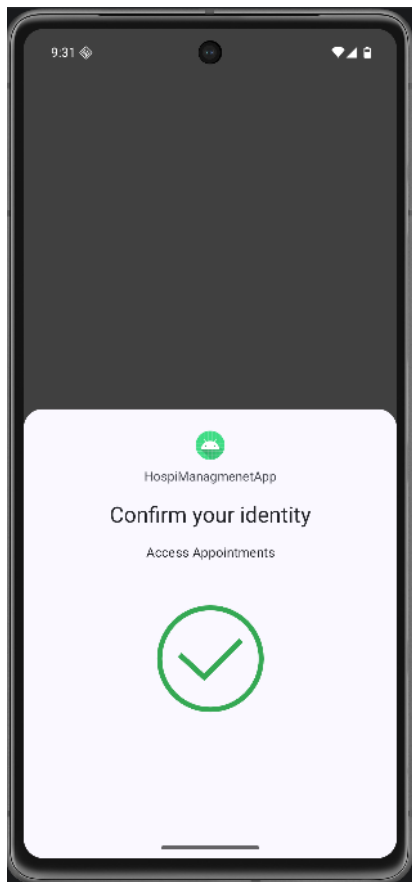


Figure 18 Biometric scan success

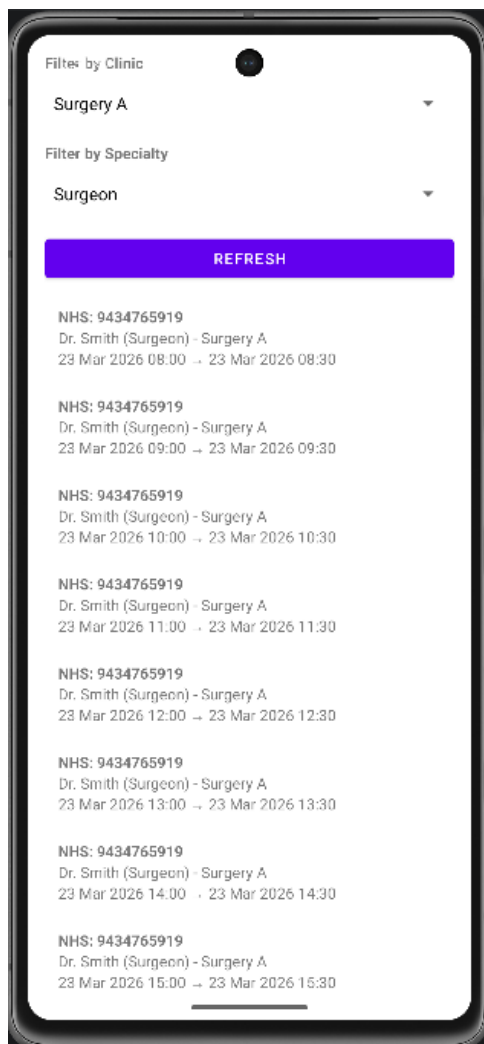


Figure 19 Appointment filtering

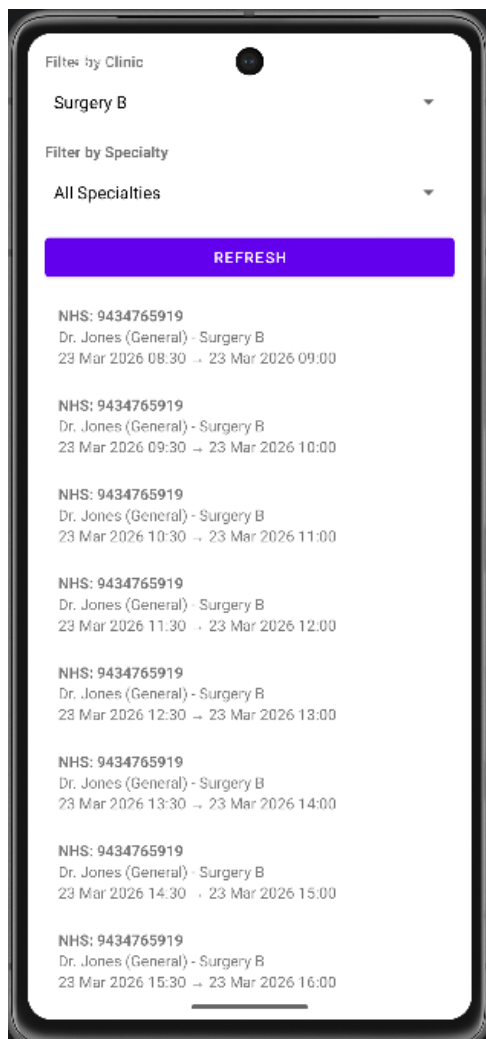


Figure 20 Appointment filtering

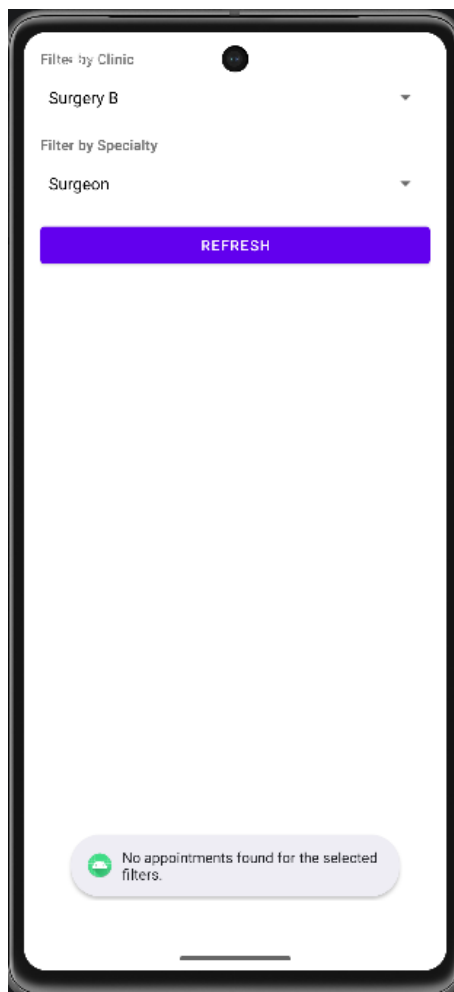


Figure 21 Appointment filtering

Filter by Clinic

All Clinics

Filter by Specialty

All Specialties

REFRESH

NHS: 9434765919

Dr. Smith (Surgeon) - Surgery A

23 Mar 2026 08:00 → 23 Mar 2026 08:30

NHS: 9434765919

Dr. Smith (Surgeon) - Surgery A

23 Mar 2026 08:00 → 23 Mar 2026 08:30

NHS: 9434765919

Dr. Smith (Surgeon) - Surgery A

23 Mar 2026 08:00 → 23 Mar 2026 08:30

NHS: 9434765919

Dr. Jones (General) - Surgery B

23 Mar 2026 08:30 → 23 Mar 2026 09:00

NHS: 9434765919

Dr. Smith (Surgeon) - Surgery A

23 Mar 2026 09:00 → 23 Mar 2026 09:30

NHS: 9434765919

Dr. Jones (General) - Surgery B

23 Mar 2026 09:30 → 23 Mar 2026 10:00

NHS: 9434765919

Dr. Smith (Surgeon) - Surgery A

23 Mar 2026 09:30 → 23 Mar 2026 10:00

NHS: 9434765919

Dr. Jones (General) - Surgery B

23 Mar 2026 10:30 → 23 Mar 2026 11:00

Figure 22 Add valid appointment

Clinician: Dr. Smith (Surgeon)

9434765919

Start Date and Time

23 Mar 2026 08:00

PICK DATE

PICK TIME

End Date and Time

23 Mar 2026 08:30

PICK DATE

PICK TIME

CONFIRM

 Time conflict detected. Please choose another slot.

Figure 23 Add invalid appointment

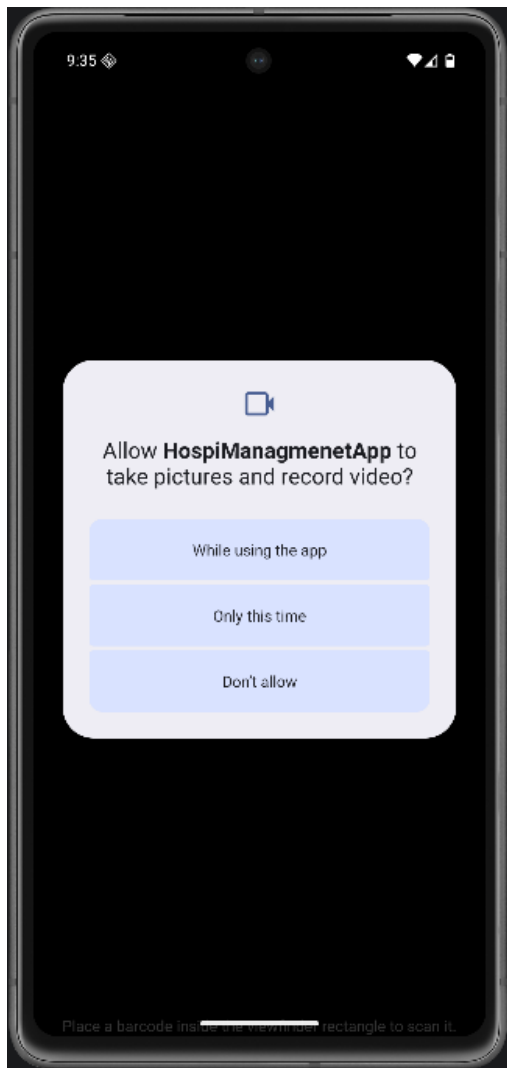


Figure 24 Barcode scanner on emulator



Figure 25 Emulated scene

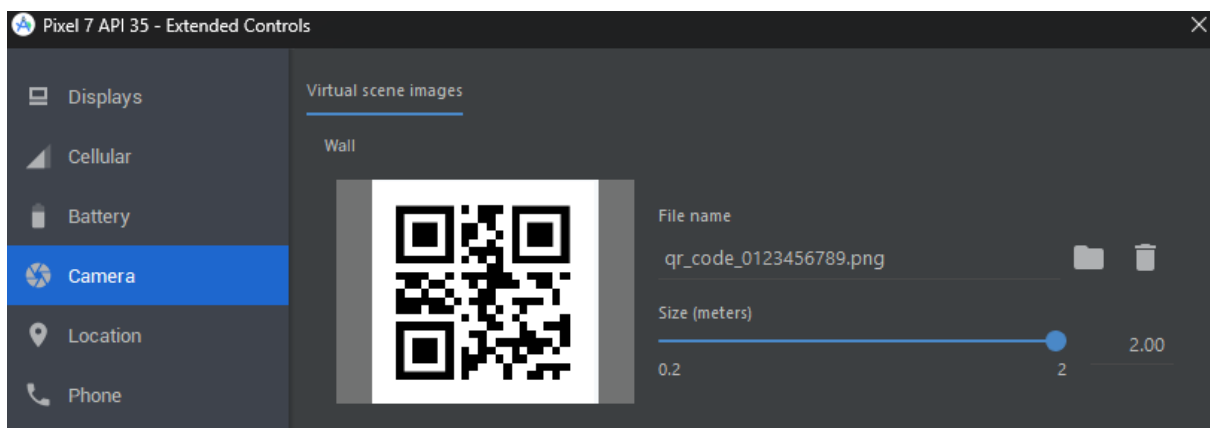


Figure 26 QR for "0123456789"



Figure 27 QR in emulated scene

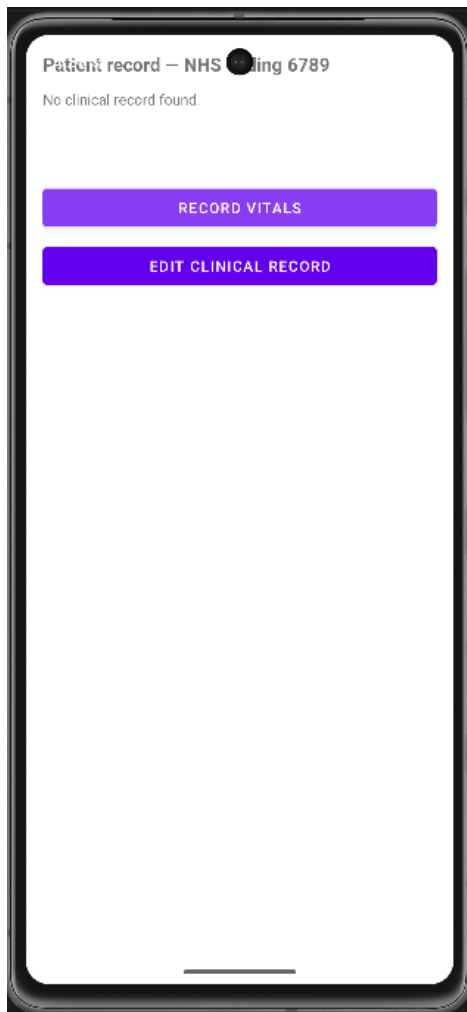


Figure 28 barcode scanner success

Edit Clinical Record

Problems / Conditions

Allergies

Medications

SAVE RECORD

Figure 29 Edit patient summary

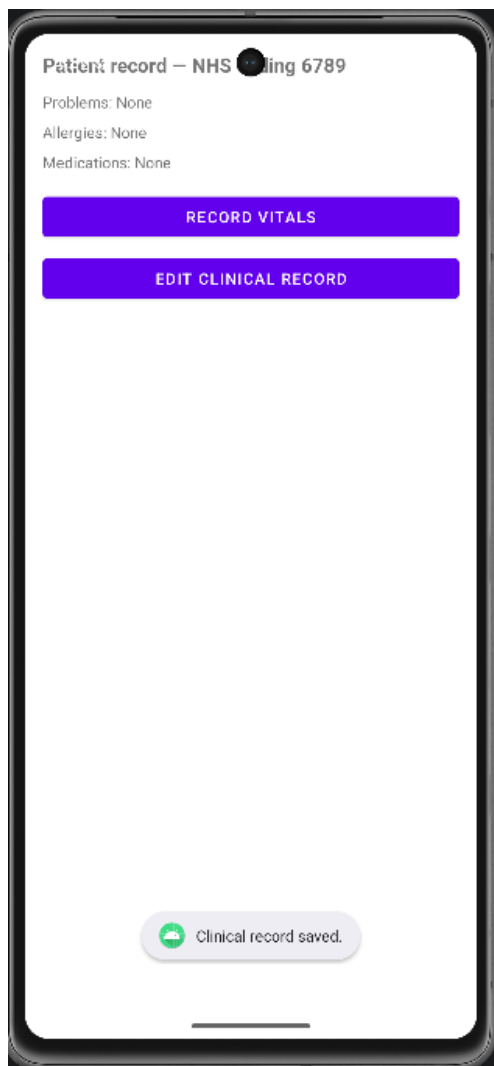


Figure 30 Clinical record change success

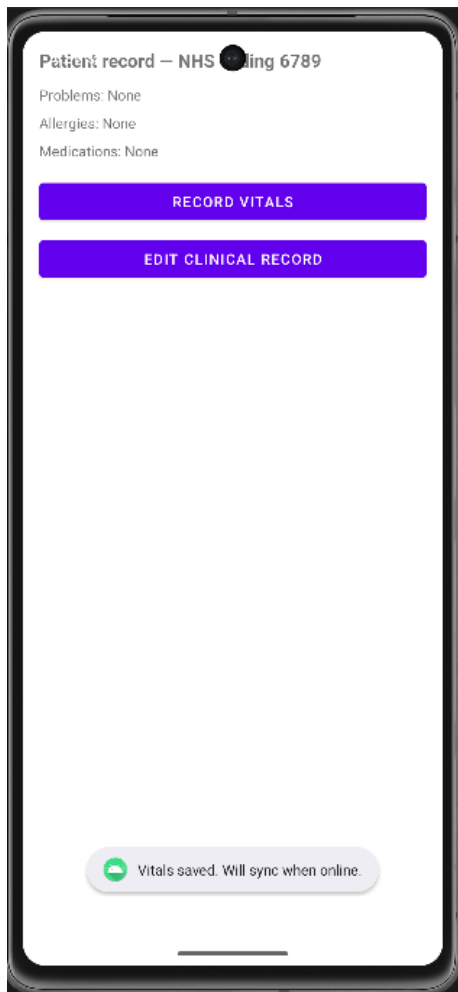


Figure 31 Vitals change WorkManager sync

↺ ☒ Live updates ↗ Results are read-only ⏪ < 50 > ⏩									
	🔗 id	↕ patientNhs	↕ temperature	↕ heartRate	↕ systolic	↕ diastolic	↕ timestamp	↕ synced	
1	1	0123456789	37.0	37	37	37	1774256251708	0	

Figure 32 Vitals

↺ ☒ Live updates ↗ Results are read-only ⏪ < 50 > ⏩									
	🔗 id	↕ nhsNumber	↕ fullName	↕ dateOfBirth	↕ phone	↕ email	↕ createdAt	↕ updatedAt	
1	1	0123456789	a patient	01-01-2000	07770184456	patient@admin.	1774256232944	1774256232944	
2	2	9434765919	Full Name	01-01-2001	07770124455	full.name@admi	1774257347959	1774257347959	

Figure 33 Patients

	🔗 id	↕ patientNhs	↕ allergies	↕ medications	↕ problems	↕ updatedAt	
1	1	0123456789	None	None	None	1774256264641	

Figure 34 Clinical records

	☒ Live updates ↗	Results are read-only						50			
	id	fullName	email	role	adminPin						
1	1	an admin	admin@admin.com	ADMIN	03ac674216f3e15c761ee1						

Figure 35 Staff

	☒ Live updates ↗	Results are read-only						50			
	id	patientNh...	startTime	endTime	clinicianId	clinicianN...	clinic	status			
1	1	9434765919	1774252800000	1774254600000	101	Dr. Smith (Surg	Surgery A	BOOKED			
2	2	9434765919	1774254600000	1774256400000	102	Dr. Jones (Gen	Surgery B	BOOKED			
3	3	9434765919	1774256400000	1774258200000	101	Dr. Smith (Surg	Surgery A	BOOKED			
4	4	9434765919	1774258200000	1774260000000	102	Dr. Jones (Gen	Surgery B	BOOKED			
5	5	9434765919	1774260000000	1774261800000	101	Dr. Smith (Surg	Surgery A	BOOKED			
6	6	9434765919	1774261800000	1774263600000	102	Dr. Jones (Gen	Surgery B	BOOKED			
7	7	9434765919	1774263600000	1774265400000	101	Dr. Smith (Surg	Surgery A	BOOKED			
8	8	9434765919	1774265400000	1774267200000	102	Dr. Jones (Gen	Surgery B	BOOKED			

Figure 36 Appointments

```
public class RbacPolicyEvaluator {

    // Simple matrix: ADMIN & RECEPTION & CLINICIAN can view list; only ADMIN & RECEPTION can
    1 usage
    public static boolean canViewAppointments(Context ctx) {
        String role = SessionManager.getCurrentRole(ctx);
        return "ADMIN".equals(role) || "RECEPTION".equals(role) || "CLINICIAN".equals(role);
    }

    1 usage
    public static boolean canBookOrReschedule(Context ctx) {
        String role = SessionManager.getCurrentRole(ctx);
        return "ADMIN".equals(role) || "RECEPTION".equals(role);
    }
}
```

Figure 37 RBAC policy

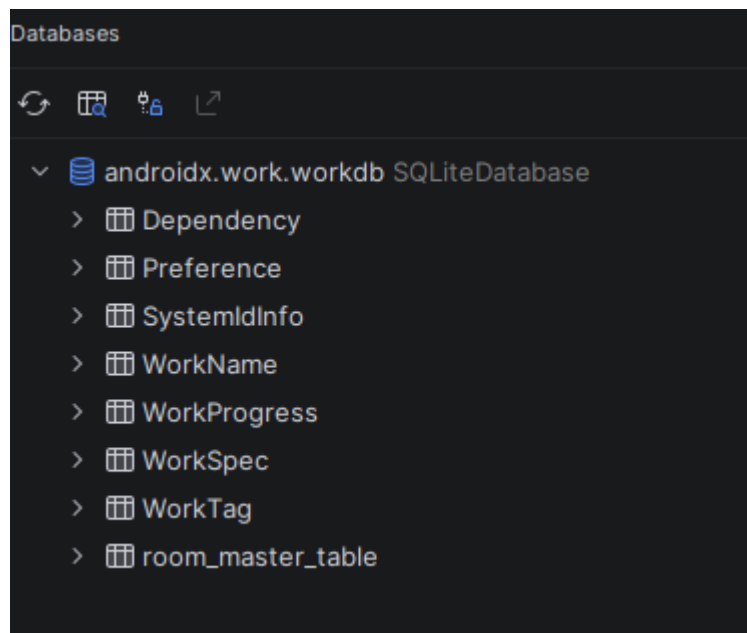


Figure 38 Encrypted DB. App Inspector shows no hms_db

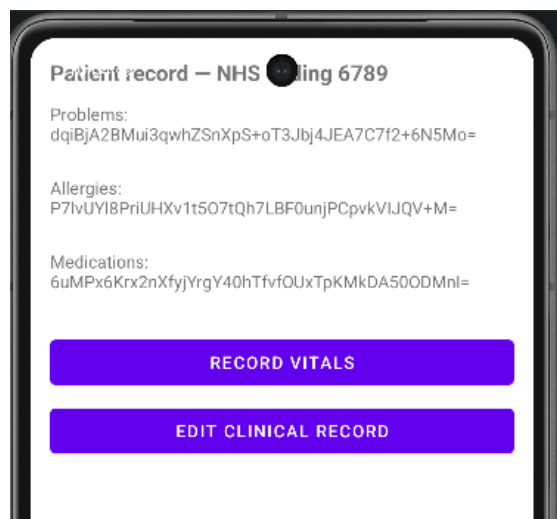


Figure 39 Encrypted data shwon on screen

```

54     public static AppDatabase getInstance(Context context) {
55         if (INSTANCE == null) {
56             synchronized (AppDatabase.class) {
57                 if (INSTANCE == null) {
58                     // SQLiteDatabase.loadLibs(context);
59                     // byte[] passphrase = SQLiteDatabase
60                     //     .getBytes("hms_secure_key_lab4".toCharArray());
61                     // SupportFactory factory = new SupportFactory(passphrase);
62
63                     INSTANCE = Room.databaseBuilder(
64                         context.getApplicationContext(),
65                         AppDatabase.class,
66                         name: "hms_db"
67                     )
68                     // .openHelperFactory(factory)
69                     .fallbackToDestructiveMigration()
70                     .build();
71                 }
72             }
73         }
74         return INSTANCE; // Return the shared database
75     }
76 }

```

Figure 40 SQL encrypting commented out

```

private void loadRecord() {
    Executors.newSingleThreadExecutor().execute(() -> {
        ClinicalRecord record = AppDatabase.getInstance(getApplicationContext())
            .clinicalRecordDao().findByPatient(nhsNumber);

        runOnUiThread(() -> {
            if (record != null) {
                // decrypt fields before display
                String problems = securityAgent.decrypt(record.problems);
                String allergies = securityAgent.decrypt(record.allergies);
                String medications = securityAgent.decrypt(record.medications);

                tvProblems.setText("Problems: " + problems);
                tvAllergies.setText("Allergies: " + allergies);
                tvMedications.setText("Medications: " + medications);
            } else {
                tvProblems.setText("No clinical record found.");
                tvAllergies.setText("");
                tvMedications.setText("");
            }
        })
    })
}

```

Figure 41 Decryption (not working)

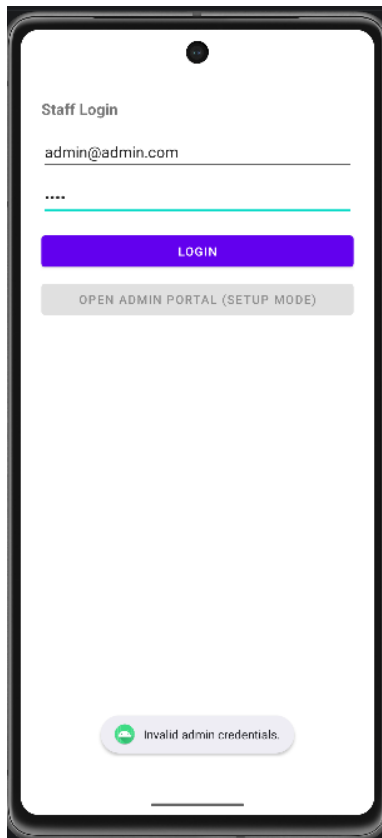


Figure 42 Invalid login leads to account lock

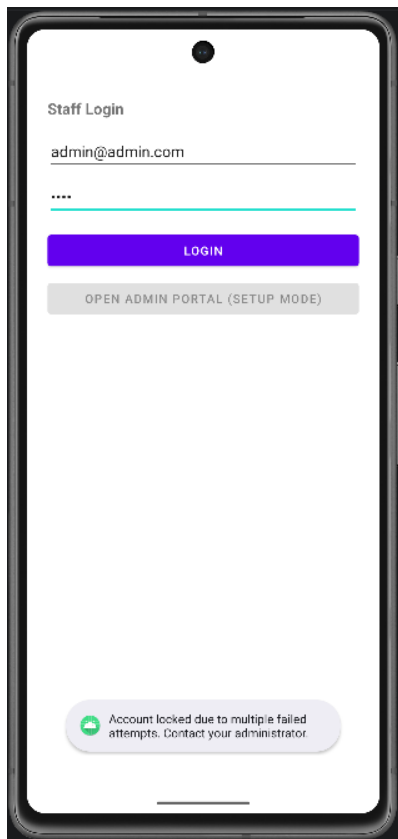


Figure 43 Account lock out message